Smart Crop Advisory System (SCA)

Working System Architecture, Environment Specifications & Production Deployment Manual

PRODUCTION LIVE (VERCEL) | RENDER API ONLINE | DOCKER MULTI-STAGE | NEON POSTGRESQL | RAM < 360 MB

1. Operational System Architecture & Live Endpoints

The **Smart Crop Advisory System (SCA)** operates as a decoupled client-server platform engineered for zero-cost, memory-constrained cloud environments:

```
[CLIENT LAYER: React 18 + Vite SPA on Vercel]
   │  https://ai-smart-farming-system-ivory.vercel.app
   │  Tri-lingual (English | ಕನ್ನಡ | हिन्दी) • Leaflet GIS Map • SVG 7-Day Price Forecast
   │
   ▼ (HTTPS / JSON REST API)
[API LAYER: FastAPI + Uvicorn ASGI on Render Free Tier (Docker)]
   │  https://ai-smart-farming-system-if62.onrender.com/health
   │  Footprint: < 360 MB active RAM (within 512 MB cgroup limit) via ONNX INT8 + malloc_trim
   │
   ├── [VMamba VSSM Tiny INT8] ─── 58-Class Leaf Pathology Engine (27.02 MB, 244 ms CPU)
   ├── [Weather-Aware RF] ──────── 19-Feature Mandi Price Regressor + Open-Meteo API Sync
   ├── [Agro-Climatic RF] ──────── 11-Feature Crop Yield Predictor (3,000 DES/ICRISAT records)
   └── [Biosecurity Service] ────── Groq LLM (llama-3.3-70b) + ICAR Chemical Schedules
   │
   ▼ (SQL Alchemy Abstract Connector)
[PERSISTENCE: Neon Serverless PostgreSQL / Local SQLite]
   Stores users (FastAuth PBKDF2), community disease alerts, upvotes, and taluk chat.
```

> **Live Deployment Directory & Credentials**
> - **Production Frontend:** https://ai-smart-farming-system-ivory.vercel.app
> - **Production Backend API:** https://ai-smart-farming-system-if62.onrender.com/health
> - **GitHub Repository:** https://github.com/mdkhalanderk/ai-smart-farming-system.git
> - **Demo Farmer Account:** Email: `test_farmer@krishi.in` | Password: `farmer123`

2. Local Development & Testing Instructions

2.1 Prerequisites

- **Python:** Version 3.10, 3.11, or 3.12 with `pip`
- **Node.js:** Version 18 or 20+ with `npm`
- **Git:** Installed and configured

2.2 Step 1 — Clone Repository & Setup Virtual Environment

```
git clone https://github.com/mdkhalanderk/ai-smart-farming-system.git
cd ai-smart-farming-system

# Create and activate virtual environment
python -m venv venv
# Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# Linux / macOS:
source venv/bin/activate

# Install backend dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

2.3 Step 2 — Configure Environment Variables

Create a `.env` file in the project root directory:

```
PORT=8000
HOST=127.0.0.1
DEBUG=True
JWT_SECRET=krishi_drishti_jwt_secret_2026
# Optional: Groq LLM API Key (uses deterministic ICAR templates if omitted)
GROQ_API_KEY=your_groq_api_key_here
# Optional: Neon PostgreSQL connection string (falls back to local SQLite if omitted)
# DATABASE_URL=postgresql://user:password@ep-cold-pool.neon.tech/neondb?sslmode=require
```

2.4 Step 3 — Launch Backend & Frontend Servers

Terminal 1: FastAPI Backend

```
python -m uvicorn backend.app:app --host
127.0.0.1 --port 8000 --reload
```

Verify via browser: `http://127.0.0.1:8000/health`

Terminal 2: React Vite Frontend

```
cd frontend
npm install
npm run dev
```

Open browser: `http://localhost:5173`

3. Production Cloud Deployment: Step-by-Step

Strategy A: Split Cloud Architecture (Vercel Frontend + Render Backend) — Active Production

Step A: Backend Deployment on Render (Docker Runtime)

1. Navigate to Render Dashboard and click **New +** → **Web Service**.
2. Connect your repository `ai-smart-farming-system`.
3. Configure service settings:
   - **Name:** `ai-smart-farming-system-api`
   - **Runtime: Docker** (Render reads the production `Dockerfile` at the repository root).
   - **Instance Plan: Free** (0.1 CPU, 512 MB RAM).
   - **Health Check Path:** `/health`
4. In **Environment Variables**, configure:

Key	Value	Purpose
`PORT`	`8000`	Uvicorn ASGI binding port
`APP_DATA_DIR`	`/app/storage`	Local persistent cache and photo upload directory
`LOW_MEMORY_MODE`	`true`	Enforces single-thread inference and glibc arena trimming
`MALLOC_ARENA_MAX`	`2`	Limits C++ glibc memory pools, preventing heap fragmentation
`ALLOWED_ORIGINS`	`https://ai-smart-farming-system-ivory.vercel.app`	CORS security header matching your frontend domain
`DATABASE_URL`	`postgresql://...neon.tech/neondb?sslmode=require`	Serverless PostgreSQL connection (optional)

5. Click **Create Web Service**. Verify live status at `https://your-api.onrender.com/health`.

Step B: Frontend Deployment on Vercel

1. Go to Vercel Dashboard → **Add New...** → **Project**.
2. Import the `ai-smart-farming-system` GitHub repository.
3. Set **Root Directory** to `frontend`.
4. Framework Preset: `Vite` | Build Command: `npm run build` | Output: `dist`.
5. Set Environment Variable:
   - `VITE_API_BASE_URL` = `https://ai-smart-farming-system-if62.onrender.com` *(without trailing slash)*
6. Click **Deploy**. Vercel automatically deploys with SPA rewrites via `frontend/vercel.json`.

Step C: Keep-Alive Polling (Eliminating 45s Cold Starts)

Render's free tier spins down instances after 15 minutes of inactivity. Set up a free monitor at UptimeRobot.com:

- **Monitor Type:** HTTP(s) | **Interval:** Every 5 minutes
- **URL:** `https://ai-smart-farming-system-if62.onrender.com/health`
- This maintains active memory state and ensures instant response times for farmers.

4. Single-Server VPS Deployment (Docker Compose)

For self-hosted deployments on AWS EC2, DigitalOcean, or an on-premise Linux server:

```
# 1. Install Docker & Compose plugin on Ubuntu
sudo apt update && sudo apt install -y docker.io docker-compose-plugin

# 2. Clone repository & start full stack
git clone https://github.com/mdkhalanderk/ai-smart-farming-system.git
cd ai-smart-farming-system
docker compose -f docker-compose.prod.yml up -d --build

# 3. Verify services
docker ps
curl http://localhost:8000/health
```

- **Nginx Frontend Proxy:** Available on Port 80 (`http://<server-ip>:80`).
- **Backend Health Check:** Available on Port 8000 (`http://<server-ip>:8000/health`).
- **Persistent Storage:** Volume `krishi_storage` mapped to `/app/storage`.

5. Comprehensive Environment Variables Reference

Variable Name	Component	Required	Default Value	Functional Description
`PORT`	Backend	No	`8000`	Network listening port for Uvicorn ASGI server.
`APP_DATA_DIR`	Backend	No	`./storage`	Filesystem location for uploads and SQLite database.
`LOW_MEMORY_MODE`	Backend	Recommended	`true`	Forces ONNX single-thread execution and aggressive C garbage collection.
`MALLOC_ARENA_MAX`	Backend	Recommended	`2`	Configures glibc to limit allocation pools, preventing RAM leakage.
`ALLOWED_ORIGINS`	Backend	Production	`*`	CORS allowed frontend domain whitelist.
`DATABASE_URL`	Backend	Optional	`sqlite:///farmer_auth.db`	Neon PostgreSQL connection URI with SSL parameter.
`GROQ_API_KEY`	Backend	Optional	`None`	API key for Groq Llama-3.3-70b outbreak bulletin synthesis.
`JWT_SECRET`	Backend	Recommended	`secret`	Cryptographic HMAC key for FastAuth JWT session tokens.
`VITE_API_BASE_URL`	Frontend	Production	`http://localhost:8000`	Base URL pointing React client to the FastAPI backend.

6. Production Verification Checklist

✓ **Backend Healthcheck:** `GET /health` returns `{"status":"healthy","model_loaded":true}`.

✓ **Vision Model Active:** Image upload on `/detect` returns diagnosis and top-3 probabilities within 300 ms.

✓ **OOD Blur Rejection:** Submitting a blurred photo returns error: "*Image is too blurry*" (Laplacian $\sigma^2 < 12$).

✓ **Mandi Forecasting:** `/price-prediction` renders 7-day SVG price curves and market regime status (High/Medium/Low).

✓ **Agro-Climatic Yield:** `/yield-prediction` calculates expected tonnes/hectare and total harvest based on acreage.

✓ **Biosecurity Syndication:** Outbreak alerts appear on Leaflet GIS map with Groq LLM advisory bulletin.

✓ **Tri-Lingual Localization:** Instant toggle between English, ಕನ್ನಡ (Kannada), and हिन्दी (Hindi).

✓ **FastAuth Security:** Password hashing via PBKDF2-HMAC-SHA256 with JWT bearer token authentication.

7. Project Credentials & Institutional Affiliation

Project Title	Smart Crop Advisory System (Precision Indian Agriculture Platform)
Department & Degree	Department of Artificial Intelligence and Machine Learning • Bachelor of Engineering (B.E.)
Institution & University	CMR Institute of Technology (CMRIT), AECS Layout, ITPL Main Road, Bengaluru – 560037 Affiliated to Visvesvaraya Technological University (VTU), Belagavi (2025–2026)
Project Guide	Prof. Laya N, Assistant Professor, Department of AIML, CMRIT
Head of Department	Dr. Shyam P Joy, Professor & Head, Department of AIML, CMRIT
Principal	Dr. Sanjay Jain, Principal, CMRIT
Student Team	1. **Navneet Gowda GT** (1CR23AI071) • 2. **Sachinkumar Narayan Naik** (1CR24AI407) 3. **Jayanth Gowda CR** (1CR23AI041) • 4. **Mohammed Khalander K** (1CR23AI062)